



# Qiskit Fall Fest PNC 2025

Nov 9-19

In collaboration with IBM Quantum, PAS, and Prithvi Narayan Campus

# QNRG

## Quantum Number Random Generator

Manuel Cañete

Nov 2025

# Outline

- **Executive Summary**
- **Functions**
- **Quantum Circuit**
- **Testing**
- **Histogram**
- **Real application example**
- **Conclusions and insights**

# Executive Summary

- **Problem**

- Common random number generator uses an initial seed plus an algorithm to calculate a pseudo-random numbers. The result is a deterministic series and not truly random.

- **Solution**

- Using a quantum circuit, we can achieve truly randomness and overcome that issue.
- The solution has been divided into two functions. This allows more flexibility in the future as we can keep the function *qrng()* as it and define new functions to fulfil new requirements (float numbers, exclude limits, list choice...).
- Function *qrng()* utilizes a quantum circuit to generate random numbers.
- Function *quantum\_randint()* adapts outcome to the requirements. In this case, integers within a range, including limits.

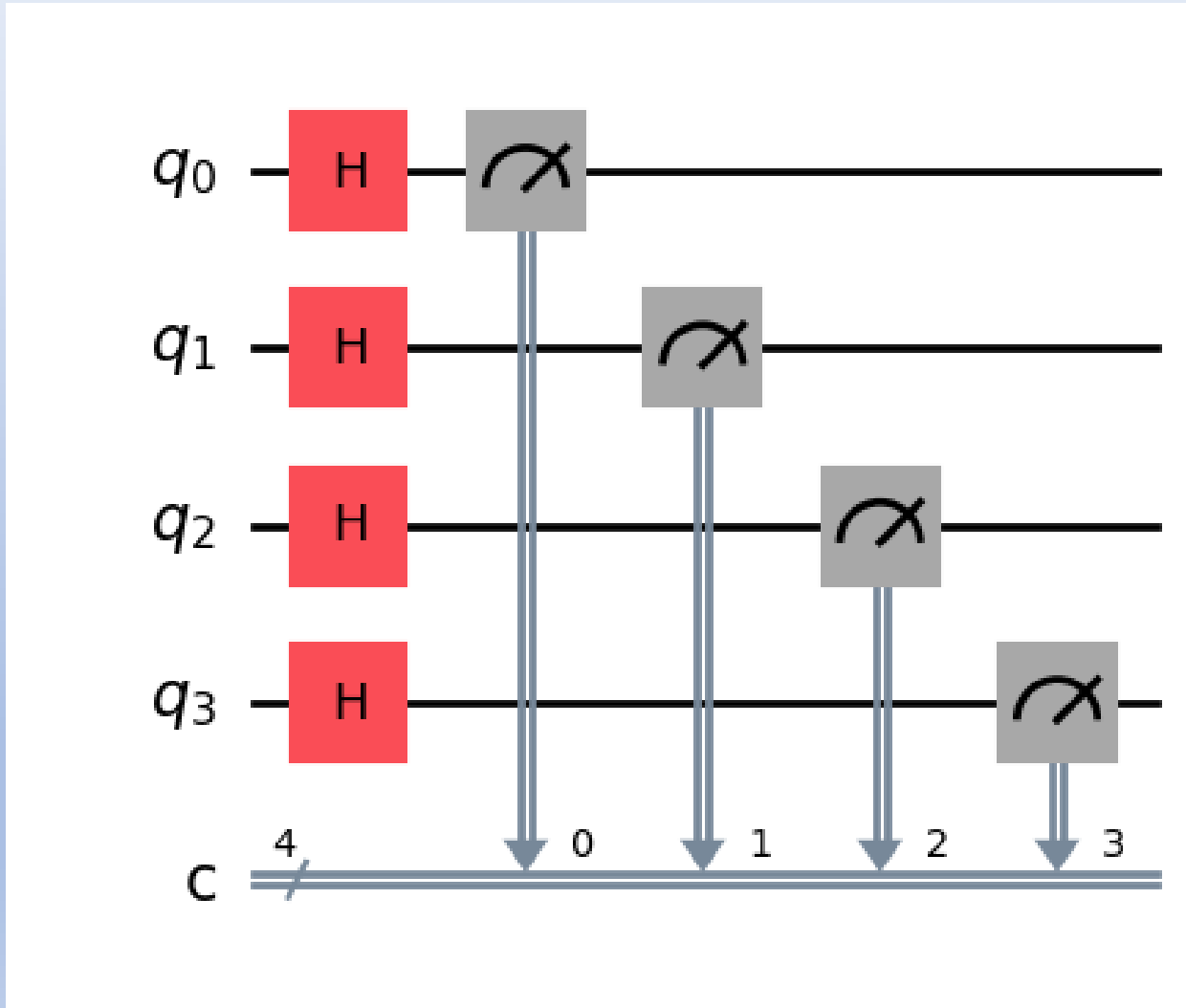
# Function *qrng()*

```
6  '''
7  qrng: Quantum Random Number Generator
8      Generates a random integer in the range [0, 2^num_qubits - 1]
9      input: num_qubits (int) - number of qubits to use
10     output: int - random integer
11  '''
12  def qrng(num_qubits: int) -> int:
13      Q_list = list(range(num_qubits))
14
15      # Define quantum circuit with 'num_qubits' qubits and 'num_qubits'
16      qc = QuantumCircuit(num_qubits, num_qubits)
17
18      # Apply Hadamard gate for each qubit
19      qc.h(Q_list)
20
```

```
20
21     # Measure qubits and store results in classical bits
22     qc.measure(Q_list, Q_list)
23
24     # Executes the circuit on the qasm simulator
25     backend = Aer.get_backend('aer_simulator')
26     tcirc = transpile(qc, backend)
27     job = backend.run(tcirc, shots=1)
28     result = job.result()
29     counts = result.get_counts()
30
31     # Get the bitstring result and convert to integer
32     bitstring = list(counts.keys())[0]
33     value = int(bitstring, 2)
34     return value
```

0.0s

# Example of quantum circuit (4 qubits)



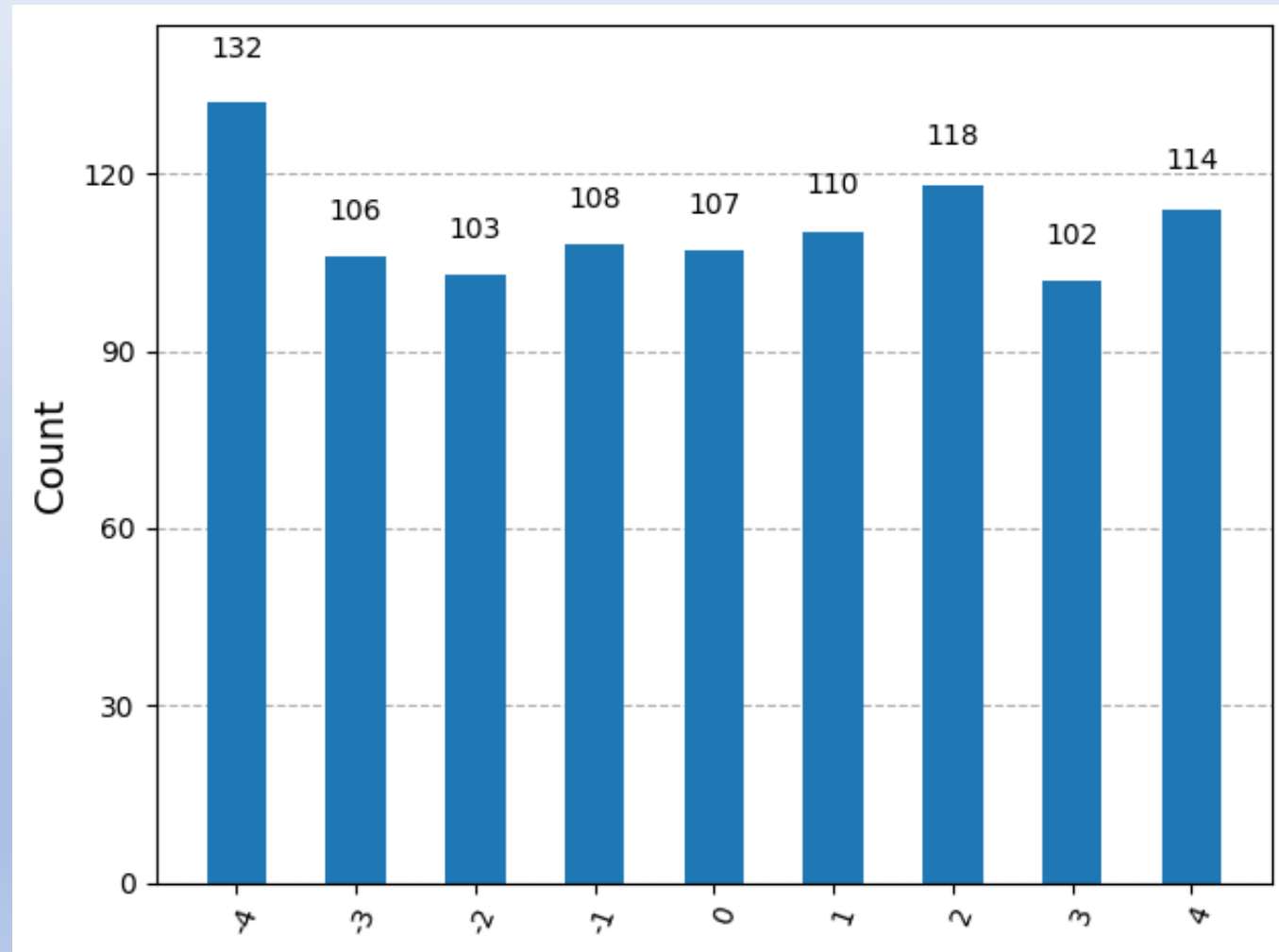
# Function *quantum\_randint()*

```
38 '''
39 quantum_randint: Generates a random integer in the range [min, max] using the function 'qnrng'
40 |   input: min (int) - minimum value (inclusive)
41 |   |   max (int) - maximum value (inclusive)
42 |   output: int - random integer in the range [min, max]
43 '''
44 def quantum_randint(min:int, max:int) -> int:
45     # Calculate the span and determine minimum number of qubits needed
46     span = max - min + 1
47     num_qubits = 1
48     while (2**num_qubits < span):
49         num_qubits += 1
50
51     # Generate random value until it falls within the desired span
52     value = qnrng(num_qubits)
53     while (value >= span):
54         value = qnrng(num_qubits)
55
56     # Adjust value to the input range
57     value += min
58
59     return value
60
```

# Testing

```
3 # Generate 1000 random integers in the range [-4, 4] and plot the histogram
4 min, max = -4, 4
5 values = [quantum_randint(min, max) for _ in range(1000)]
6
7 # Count occurrences of each value for histogram plotting (list to dict)
8 counts = {}
9 for v in values:
10 |     counts[v] = counts.get(v, 0) + 1
11
12 # Plot the histogram
13 plot_histogram(counts)
```

# Histogram



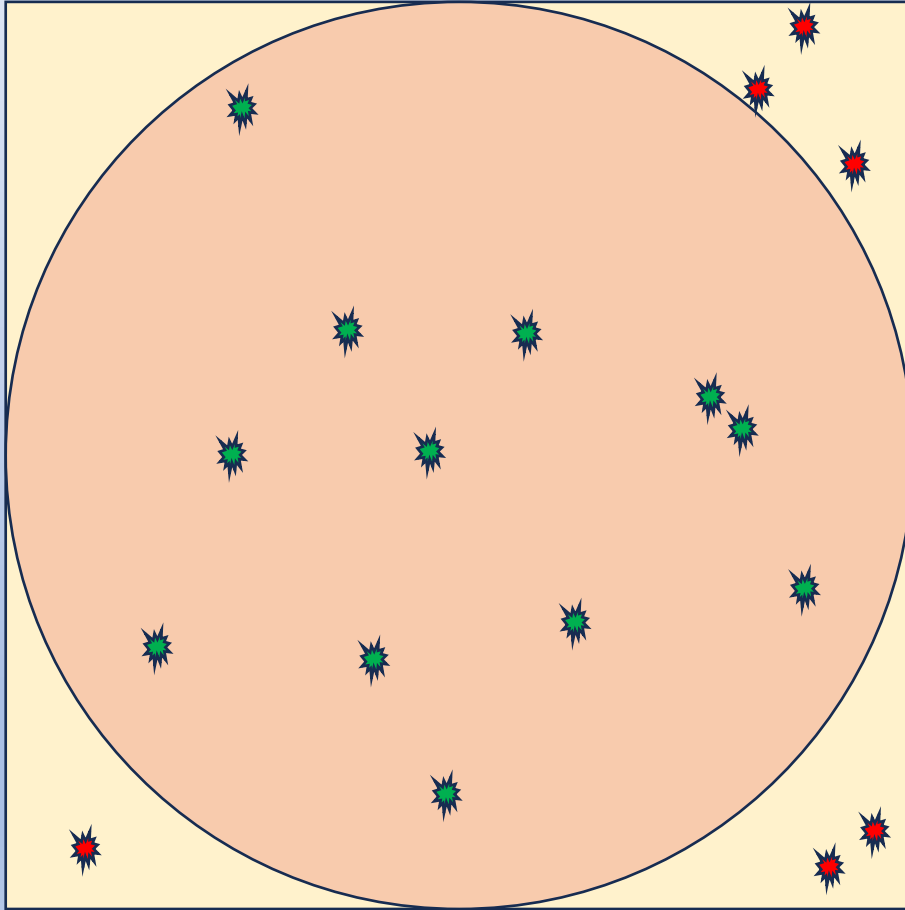
# Real application: Montecarlo simulation

- Real application: calculate value of  $\pi$  using random numbers
  - Consider a circle of radius  $R$  inside a square of side  $2*R$ :
  - Area of the square =  $(2*R)^2$
  - Area of the circle =  $\pi * R^2$
  - Areas\_ratio =  $4 / \pi \rightarrow \pi = 4 * \text{Areas\_ratio}$
- Generate random numbers, check if inside the circle:
  - Generate random numbers between  $[-R, R]$ , inside the square  $\rightarrow$  total\_hits
  - Check and count number of hits inside the circle  $\rightarrow$  inside\_hits
  - Ratio of total\_hits vs. inside\_hits should be same as the ratio between areas
  - Estimation of  $\pi = 4 * (\text{inside\_hits} / \text{total\_hits})$

# Real application: Montecarlo simulation

```
1 # Initialize parameters for Montecarlo simulation
2 radius = 100
3 total_hits = 10*radius
4 inside_hits = 0
5
6 # Perform Montecarlo simulation using quantum random numbers to estimate value of pi
7 # For each hit, generate random (x, y) coordinates and check if they fall inside the circle
8 for _ in range(total_hits):
9     x = quantum_randint(-radius, radius)
10    y = quantum_randint(-radius, radius)
11    if x**2 + y**2 <= radius**2:
12        inside_hits += 1
13
14 # Estimate the value of pi as four times the ratio of hits inside the circle to total hits
15 number_pi = 4 * (inside_hits / total_hits)
16 print(f"Estimated value of pi: {number_pi}")
17
```

# Real application: Montecarlo simulation



$$\pi = 4 * (\text{green star} / \text{red star})$$

# Conclusions and Insights

- Quantum circuit can generate truly and non-deterministic random numbers, overcoming the problem of pseudo-random numbers algorithms.
- Quantum random numbers can be used in a wide range of applications. Estimation of number  $\pi$  is a simple demonstration of the possibilities.
- Scalability of the solution is fully assured:
  - Function *qrng()* can work with any number of qubits.
  - New functions based on *qrng()* can be defined to adapt outcomes to necessities.